Field Hours

CSCE 627 Theory of Computability

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

A stack for parentheses

You recognize balanced parentheses with a pushdown automaton: one state, one stack symbol X, and a bottom marker Z. On an open parenthesis you push X. On a close you pop X, and you reject if the top is not X. You accept at the end only when Z is on top. The input (()) pushes to height 2 above Z and then pops back to Z. The stack is the whole memory.

You walk (()) one symbol at a time. After the first open the stack is Z X. After the second open it is Z X X. The first close leaves Z X, and the second close leaves Z, so you accept. The string (() ends with Z X still on the stack, so you reject even though every pop succeeded. Length 3 is enough to show both failures: a bad pop, and a nonempty stack at the end.

Stack while reading (())

1. Push to stack Z X
2. Push to stack Z X X
3. Pop back to Z X
4. Pop to Z and accept

Accepting (())

1. Start with Z alone. Two opens produce Z X X, height 2 above the marker.
2. Two closes pop both X symbols and leave Z.
3. The end test sees Z and accepts. The machine used one state and one stack.

Problem 1. Trace () on this machine. Does it accept, and what is the maximum height above Z?

Answer. One open pushes X, height 1. One close pops it. The stack is Z, so you accept. The maximum height above Z is 1.

Problem 2. Trace the input ()). Where does the machine reject?

Answer. The open pushes X. The first close pops it and leaves Z. The second close sees Z rather than X and rejects.

Lecture 2 · 90 minutes

The grammar that matches the stack

You generate the same strings with a grammar. The productions are S goes to (S) S, and S goes to the empty

string. From S you rewrite to (S) S, then rewrite the inner S the same way, then erase every remaining S. The yield is (()). That derivation and the one-state machine agree on this string of length 4. The grammar builds the string. The automaton checks it.

You can see why ()) has no derivation. Each use of S goes to (S) S adds one open and one close, and the empty production adds neither, so the two counts stay equal. The string ()) has 1 open and 2 closes. No sequence of productions yields it, which matches the stack machine rejecting it. One nonterminal is enough here because every unfinished suffix is another copy of S.

Derive (()) from S

S becomes (S) S -> Inner S grows a pair -> Erase leftover S symbols -> Yield is (())

Three rewrites to (())

1. Start at S and apply S goes to (S) S.

2. Apply that production again to the first S, producing ((S) S) S.

3. Erase every S. The terminals left are (()). The stack machine accepts this same string of length 4.

Problem 1. Derive () with these productions. How many uses of S goes to (S) S do you need?

Answer. One. S goes to (S) S, then both leftover S symbols become empty, yielding (). You do not need the second nesting step.

Problem 2. Does the grammar generate ())?

Answer. No. Each parentheses production adds one open and one close, so the counts stay equal. The string ()) has 2 closes and 1 open.

Lecture 3 · 90 minutes

Pumping past the context-free sets

You prove that strings a^n b^n c^n are not context-free with the pumping lemma. The lemma gives a length p. The string with p letters of each kind splits into five pieces u, v, x, y, z, where v y is nonempty and the middle v x y has length at most p. A window that short sits in one block of letters or across 1 boundary. It cannot touch a's, b's, and c's together.

You pump by setting i = 2, which inserts one extra copy of v and of y. Those copies add letters to at most two of the three blocks, so the equal counts break. Take p = 3 and the string aaabbbccc, and let the middle be the three b's with v equal to one b. The pumped string has 3 a's, 4 b's, and 3 c's. It is outside the language. Every other window fails the same way.

One pump breaks equality

Before the pump

3 a, 3 b, 3 c

Middle sits in the b block

After i = 2

3 a, 4 b, 3 c

Counts are not equal

Pumping inside the b's

1. Let v x y be the three b's in aaabbbccc, and let v be a single b.

2. The pumped string at i = 2 is aaa, then four b's, then ccc.

3. The counts 3, 4, 3 are not equal, so the string leaves the language. The split did not stay inside.

Problem 1. Same string aaabbbccc, but v y is the boundary pair bc. What are the counts after i = 2?

Answer. You add one b and one c. The counts become 3 a's, 4 b's, and 4 c's. The a count did not rise, so the string is outside the language.

Problem 2. Why can a window of length p = 3 not contain an a, a b, and a c inside a^3 b^3 c^3?

Answer. Three b's sit between the a's and the c's. Any three consecutive letters cover a and b, or b and c, but not all three letters.

Lecture 4 · 90 minutes

A Turing machine you can step

You run a Turing machine as a table of steps: state, symbol read, symbol written, direction, next state. This machine accepts any tape that contains at least one 1. In state Scan, a 0 writes 0, moves right, and stays in Scan. A 1 writes 1, moves right, and enters Yes. A blank enters No. On the input 001 the head needs 3 steps to read the 1 and accept.

You should notice that this machine always halts. Every step moves right, the input has length 3, and the blank ends the scan. It is a decider for this one language: every input stops with yes or no. Do not promote that fact into a claim about every machine. On 001 the first two steps read 0 and stay in Scan, and the third step reads 1 and enters Yes. You can point at the row that fired.

Scan 001 until a 1

1. Read 0, move right

2. Read 0, move right

3. Read 1, enter Yes

Three steps on 001

1. Tape 0 0 1, head on the first 0, state Scan.

2. Step 1 and step 2 each read 0, write 0, move right, and stay in Scan.

3. Step 3 reads 1 and enters Yes. The machine accepts in 3 steps, the length of the input.

Problem 1. Run the same machine on input 00. What happens, and does it halt?

Answer. Two steps cross the two 0 symbols, still in Scan. The next cell is blank, so the machine enters

No and rejects. It halts. It does not accept.

Problem 2. Change the row for symbol 1 so the machine stays in Scan and moves right. Does input 001 still accept?

Answer. No. The head slides from the 1 onto the blank and then enters No. The Yes row was the only accepting step.

Lecture 5 · 90 minutes

Recursion that stays primitive

You build primitive recursive functions from zero, successor, and projections, using composition and one recursion schema. The schema sets f(x, 0) to a base value and sets f(x, y+1) from x, y, and f(x, y) only. Addition fits: add(x, 0) = x, and add(x, y+1) is the successor of add(x, y). You compute add(2, 2) as add(2, 0) = 2, then 3, then 4. Two successors take you from 2 to 4.

You cannot fit Ackermann's function into that schema, because a call increases the first argument, not only the second. It is still total, so some Turing machine computes it. Set A(0, n) = n + 1, A(m, 0) = A(m - 1, 1), and otherwise A(m, n) = A(m - 1, A(m, n - 1)). Then A(1, 0) = 2, A(1, 1) = 3, and A(1, 2) = 4. No primitive recursive definition gives the function.

add(2, 2) in two steps

add(2, 0) equals 2

add(2, 1) equals 3

add(2, 2) equals 4

Ackermann at (1, 2)

1. A(1, 0) uses the zero rule: A(0, 1) = 2.

2. A(1, 1) = A(0, A(1, 0)) = A(0, 2) = 3.

3. A(1, 2) = A(0, A(1, 1)) = A(0, 3) = 4.

Problem 1. Compute add(3, 2) with the same schema.

Answer. add(3, 0) = 3, add(3, 1) = 4, and add(3, 2) = 5. You apply successor twice to 3.

Problem 2. Compute A(2, 1) from the rules, using A(1, 1) = 3.

Answer. A(2, 0) = A(1, 1) = 3. Then A(2, 1) = A(1, 3). And A(1, 3) = A(0, A(1, 2)) = A(0, 4) = 5. So A(2, 1) = 5.

Lecture 6 · 90 minutes

Why the halting decider breaks

You cannot build a decider H that always halts and says whether machine M halts on w. From H you build D. On input M, D asks H about M running on its own text. If H says halt, D loops. If H says loop, D halts. Run D on D. Each answer H might give is the opposite of what D then does. Both of the 2 branches contradict, so no such H

exists.

You can see the flip on 2 programs. Suppose P1 halts on P1 and P2 loops on P2. D does the opposite: it loops on P1 and halts on P2. D cannot be P1, because that cell would both halt and loop, and it cannot be P2 either. The real argument lists every program, including D. You add one row, and that row breaks the same way.

Flip the two diagonal cells

What H claims

P1 on P1 halts

P2 on P2 loops

What D does

D on P1 loops

D on P2 halts

A diagonal that will not sit still

1. H claims P1 halts on its own text and P2 loops on its own text.

2. D loops on P1 and halts on P2, the opposite of that diagonal.

3. D cannot equal P1 or P2. A complete table would also need a row for D, and that row contradicts itself.

Problem 1. Suppose H says that D halts on D. What does D actually do?

Answer. If H says halt, D loops. So D does not halt, and H's yes answer is wrong. A decider is not allowed that wrong answer.

Problem 2. The Scan machine accepts 001 in 3 steps. Does that one halting run show that H exists?

Answer. No. H must answer for every machine and every input. One machine you built to halt on one tape is a question H might be asked. It is not H.

Lecture 7 · 90 minutes

Decidable, and still a long table

You call a language decidable when some Turing machine halts with the right yes or no on every input. The cost can still be unusable. Satisfiability on n boolean variables is decidable by trying every assignment, and there are 2 to the n of them. For n = 3 you try 8 rows. Each row plugs 3 bits into the formula. If any row is true you accept. If all 8 fail you reject. Adding one variable doubles the table.

You keep that search away from the halting problem. The search halts because you wrote the bound 8 into the machine before it started. Halting has no bound, written from the input length, that works for every machine. Your Scan machine halted on 001 in 3 steps because you designed the moves. A decider for one language is not a procedure that predicts arbitrary machines. You name the bound that makes the procedure halt.

Search the eight rows

Write the 3 bits -> Evaluate the formula -> Accept on a true row -> Reject if all 8 fail

The formula x and y and z

1. There are 8 assignments of three bits.

2. The row 1, 1, 1 makes every conjunct true. The other 7 rows miss at least one variable.

3. The decider accepts when it reaches that row. It would reject only after all 8 failed.

Problem 1. How many assignments do you try for n = 4, and how does that compare with n = 3?

Answer. 2 to the 4th is 16, twice the 8 rows for n = 3. One added variable doubles the search, and the procedure still halts.

Problem 2. Why does trying all 8 rows not solve the halting problem?

Answer. Those 8 rows exist because a formula in 3 variables has exactly that many assignments. A machine can run longer than any bound read off the input length. Halting has no such finite table.

Lecture 8 · 90 minutes

What you can now separate

You can now build a stack machine and a grammar for parentheses, reject a^n b^n c^n by pumping, and run a 3-step Turing machine by hand. You can also separate three grades. Parentheses are context-free. Strings with equal blocks of a, b, and c are decidable and not context-free. Halting is not decidable. For n = 2 the string aabbcc has length 6, and one left-to-right scan counts 2, 2, and 2.

You do not let that decider pretend to be H. On aabbcc the counts match and you accept. On aabbc the counts are 2, 2, and 1 and you reject. Both runs halt. H would have to answer for every machine, including the diagonal machine D. A language can sit outside the context-free languages and still have a decider that scans once. A decider for one language is not a halting oracle. You say which claim you actually proved.

Three grades, one skill

Parentheses, one stack

Equal blocks, one scan

Halting, no decider

Counting aabbcc

1. One left-to-right scan of aabbcc counts 2 a's, then 2 b's, then 2 c's.

2. The counts match, so the decider accepts and halts. On aabbc the counts are 2, 2, 1 and it rejects.

3. This decider never takes a machine description. It cannot say whether D halts on D.

Problem 1. The counts on aaabbbccc are 3, 3, and 3. Does the decider accept, and is the language context-free?

Answer. The decider accepts because the counts match. The language is still not context-free. Pumping already ruled out every grammar. Decidable is not the same as context-free.

Problem 2. Combine the stack machine with the halting diagonal. The parenthesis machine accepts (()) and halts, with stack height 2 in the middle. Why is that machine not the decider H?

Answer. H must take an arbitrary machine and input and say whether that run halts. The parenthesis machine only checks one string against a stack. The height-2 accept shows (()) is a yes for that language. It never runs D on D.